

FleetLink

מערכת משחק צוללות מרובת משתתפים מסמך תכנון ומבנה המערכת

קורס: 20503 - סדנה בתכנות מתקדם בשפת Java

מגיש: תומר גלוברמן

תוכן עניינים:

1.	מטרת המסמך. עמוד מספר 3.
2.	תיאור כללי של המערכת. עמוד מספר 3.
3.	ארכיטקטורת המערכת. עמוד מספר 4.
4.	טכנולוגיות בשימוש. עמוד מספר 6.
5.	מבנה הפרויקט והמודולים. עמוד מספר 7.
6.	מודול shared. עמוד מספר 8.
7.	תקשורת RMI. עמוד מספר 9.
8.	מבנה השרת. עמוד מספר 11.
9.	ניהול משתמשים ו-Sessions. עמוד מספר 13.
10.	Matchmaking. עמוד מספר 15.
11.	מנוע המשחק. עמוד מספר 16.
12.	מגבלות זמן ו-AFK. עמוד מספר 18.
13.	סיום משחק ושמירת תוצאה. עמוד מספר 20.
14.	Rematch. עמוד מספר 22.
15.	מערכת הדירוג. עמוד מספר 24.
16.	סטטיסטיקות, היסטוריה וטבלת מובילים. עמוד מספר 26.
17.	שכבת Persistence. עמוד מספר 28.
18.	בסיס הנתונים. עמוד מספר 29.
19.	ERD. עמוד מספר 32.
20.	מבנה הלקוח ו-JavaFX. עמוד מספר 33.
21.	Navigation ו-Screen Lifecycle. עמוד מספר 35.
22.	Concurrency וניהול מספר משחקים במקביל. עמוד מספר 37.
23.	קשרים בין המחלקות המרכזיות. עמוד מספר 39.
24.	מחלקות ומתודות עיקריות. עמוד מספר 40.
25.	טיפול בשגיאות ומצבים חריגים. עמוד מספר 43.
26.	בנייה והרצת המערכת. עמוד מספר 44.
27.	בדיקות ואימות. עמוד מספר 46.
28.	החלטות תכנון ומגבלות. עמוד מספר 48.
29.	סיכום. עמוד מספר 50.

1. מטרת המסמך -

מסמך זה מתאר את המבנה הטכני של מערכת FleetLink. המטרה היא להסביר כיצד המערכת בנויה, אילו טכנולוגיות משמשות אותה, כיצד הלקוח מתקשר עם השרת, כיצד מנוהלים משחקים במקביל וכיצד נשמר מידע קבוע בבסיס הנתונים. המסמך מתאר את המבנה הסופי של המערכת כפי שהוא ממומש בפרויקט, וכן את המחלקות והרכיבים המרכזיים ואת הקשרים ביניהם.

2. תיאור כללי של המערכת -

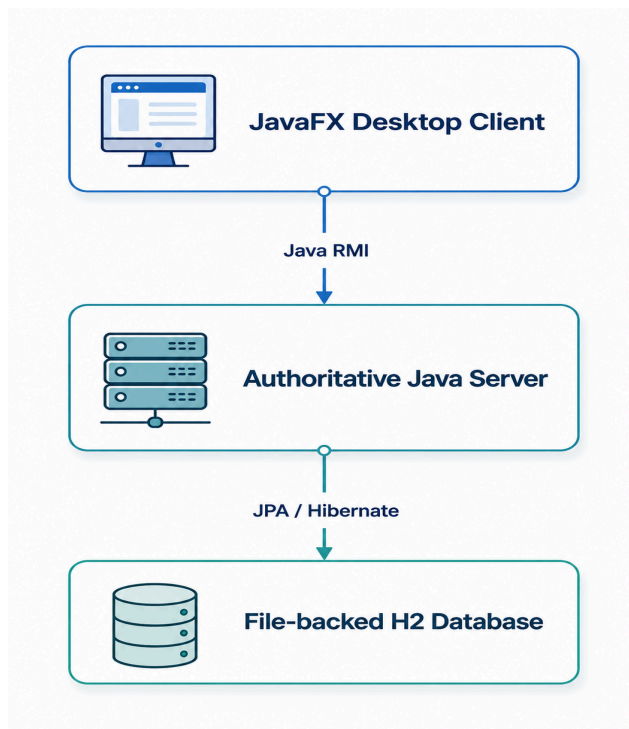
FleetLink היא מערכת Client-Server המאפשרת לזוגות של שחקנים מרוחקים לשחק זה מול זה במשחק צוללות. כל שחקן מפעיל Client שולחני עם ממשק גרפי המבוסס על JavaFX, והלקוחות מתקשרים עם שרת מרכזי באמצעות Java RMI. השרת הוא Authoritative Server, כלומר הוא מקור האמת של המשחק. ה-Client שולח פעולות שהמשתמש מבקש לבצע, אך השרת הוא זה שבודק אם הן חוקיות ומעדכן את מצב המשחק בהתאם. השרת אחראי בין היתר על:

- Sessions ו-Accounts.
- Matchmaking.
- יצירת משחקים.
- בדיקת הצבת ספינות.
- ניהול תורות.
- ביצוע יריות.
- Deadlines ו-AFK.
- סיום משחק.
- Forfeit.
- Rematch.
- Rating.
- שמירת משחקים שהסתיימו.
- Player Statistics.
- Match History.
- Leaderboard.

מידע קבוע נשמר בבסיס נתונים H2 באמצעות JPA ו-Hibernate. מצב של משחק פעיל נשמר בזיכרון של השרת ואינו נשמר בבסיס הנתונים.

3. ארכיטקטורת המערכת -

3.1 תרשים כללי

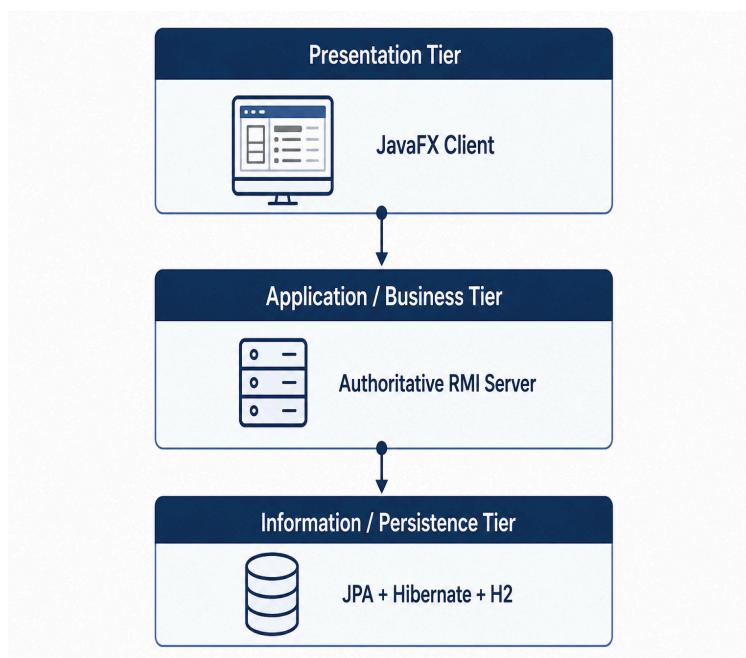


איור 1: ארכיטקטורת המערכת - מבנה ה-Client-Server והקשר בין JavaFX, שרת ה-RMI ושכבת ה-Persistence.

ה-Client אחראי על ממשק המשתמש, קבלת פעולות מהמשתמש ושליחת בקשות.
 ה-Server אחראי על הלוגיקה העסקית, חוקי המשחק ומצב המשחק.
 בסיס הנתונים אחראי על מידע שצריך להישאר קיים גם לאחר הפעלה מחדש של השרת.
 הפרדה זו מונעת מצב שבו Client יכול לקבוע בעצמו נתונים חשובים כגון תור, פגיעה, ניצחון או Rating.

3.2 הקבלה למבנה Three-Tier

בהרצאה על אפליקציות Web דיברתי על מבנה Three-Tier, שבו קיימת הפרדה בין שכבת התצוגה, שכבת הלוגיקה ושכבת המידע. FleetLink אינה אפליקציית Web, אך מבחינת חלוקת האחריות קיימת הקבלה דומה. JavaFX משמש לשכבת התצוגה, שרת ה-RMI מנהל את הלוגיקה העסקית ואת מצב המשחק, ו-JPA, Hibernate ו-H2 משמשים לשכבת ה-Persistence. ההפרדה הזאת מאפשרת לשמור את ממשק המשתמש, הלוגיקה העסקית ושמירת הנתונים כרכיבים נפרדים.



איור 2: הקבלה למבנה Three-Tier - חלוקת המערכת לשכבת תצוגה, שכבת לוגיקה ושכבת מידע.

4. טכנולוגיות בשימוש -

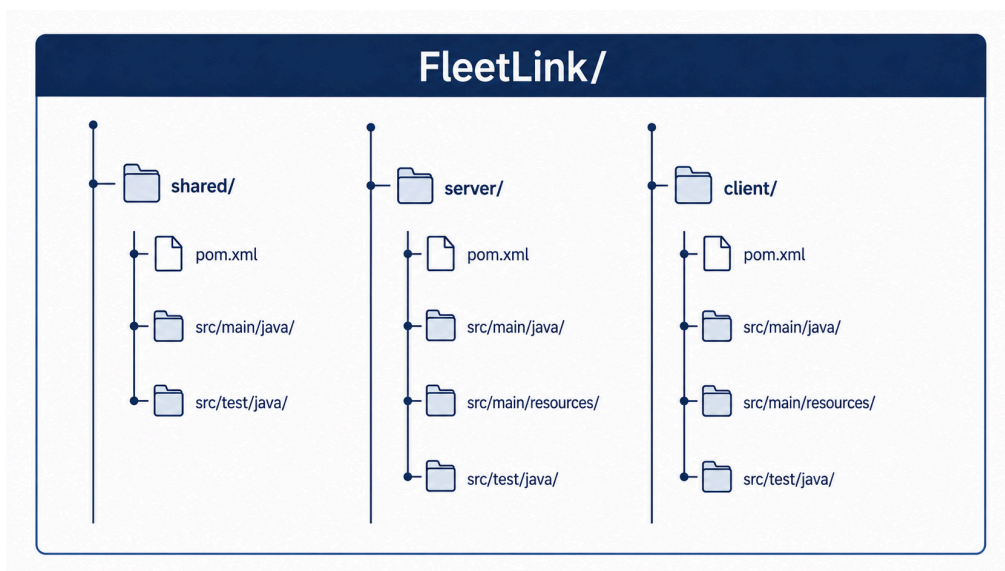
שימוש במערכת	טכנולוגיה
שפת המימוש של כל המערכת	Java 21
ממשק המשתמש של ה-Client	JavaFX 21.0.12
תקשורת Client-Server ו-Callbacks	Java RMI
API לשכבת ה-Persistence	Jakarta Persistence 3.1
מימוש JPA	Hibernate ORM 6.6.54.Final
בסיס נתונים רלציוני file-backed	H2 2.4.240
Build וניהול dependencies	Maven
בדיקות אוטומטיות	JUnit Jupiter 5.14.4

FleetLink משתמשת בשלוש טכנולוגיות מרכזיות שנלמדו במסגרת הקורס והסדנה:

- JavaFX.
- Java RMI.
- JPA-compatible Persistence.

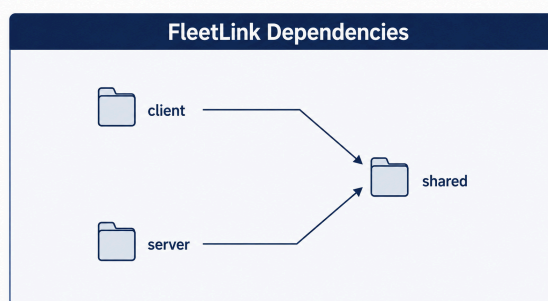
5. מבנה הפרויקט והמודולים -

הפרויקט בנוי כ-Maven multi-module project ומחולק לשלושה מודולים עיקריים: server, shared, ו-client.



איור 3: מבנה הפרויקט - חלוקת FleetLink לשלושת המודולים server, shared, ו-client.

ה-dependencies המרכזיים בין המודולים הם:



איור 4: תלויות בין המודולים - ה-client וה-server תלויים ב-shared ללא תלות ישירה ביניהם.

ה-Client אינו תלוי ב-Server. ה-Shared אינו תלוי ב-JavaFX או ב-Persistence. JPA Entities נמצאות רק ב-Server ואינן עוברות דרך RMI.

6. מודול shared -

המודול shared מכיל את החוזים והאובייקטים שצריכים להיות מוכרים גם ל-Client וגם ל-Server.
שני ה-packages המרכזיים הם:

- io.github.tomerg12.fleetlink.shared.protocol.
- io.github.tomerg12.fleetlink.shared.rmi.

protocol 6.1

ה-package מכיל DTOs, Enums ו-Value Objects שעוברים בגבול ה-RMI. מחלקות מרכזיות:

- Coordinate.
- ShipPlacement.
- GameView.
- PlayerView.
- ShotResult.
- FleetSubmissionResult.
- MatchmakingResult.
- SessionResult.
- PlayerStatisticsView.
- MatchHistoryEntryView.
- LeaderboardEntryView.
- RematchStatusView.

Enums מרכזיים:

- GamePhase.
- GameEndReason.
- ShotOutcome.
- MatchOutcome.
- RematchState.

GameView הוא Snapshot בטוח של מצב המשחק מנקודת המבט של שחקן מסוים. השרת אינו שולח לשחקן את מיקומי הספינות הסודיים של היריב.

rmi 6.2

ה-package מכיל שני Interfaces מרכזיים:

- FleetLinkServerRemote
- FleetLinkClientCallback

FleetLinkServerRemote משמש לפניות ל-Client.
FleetLinkClientCallback משמש לעדכונים ל-Server.

7. תקשורת RMI -

FleetLinkServerRemote 7.1

הפעולות המרכזיות הן:

תפקיד	Method
התחברות משתמש רשום	login
יצירת חשבון והתחברות	register
יצירת Guest Session	connectAsGuest
סיום Session	logout
כניסה לתור Matchmaking	joinMatchmaking
ביטול המתנה	cancelMatchmaking
שליחת צי מלא	submitFleet
ביצוע ירייה	fire
קבלת GameView עדכני	getCurrentGame
קבלת Statistics ו-History	getPlayerStatistics
קבלת Leaderboard	getLeaderboard
עזיבת משחק פעיל	leaveGame
בקשת Rematch	requestRematch
תשובה להזדמנות Rematch	respondToRematch

פעולות יצירת ה-Session, כלומר login, register ו-connectAsGuest, מקבלות גם FleetLinkClientCallback שה-Client ייצא באמצעות RMI. לאחר חיבור מוצלח השרת שומר את ה-Callback ומשתמש בו כדי לשלוח עדכונים ל-Client. לאחר שנוצר Session, רוב הפעולות הבאות מזוהות באמצעות sessionId. ה-Client אינו שולח בכל פעולה את ה-Rating שלו, את מספר המשחק או את זהות התור, משום שהשרת מפיק את המידע הזה מה-Session ומהמצב שהוא מנהל.

FleetLinkClientCallback 7.2

ה-Callbacks הם:

תפקיד	Method
משחק חדש נוצר	onMatchFound
מצב המשחק השתנה	onGameStateChanged
היריב ביקש Rematch	onRematchRequested
מצב ה-Rematch השתנה	onRematchStatusChanged

Callback יכול להגיע לפני שהקריאה הסינכרונית שגרמה לו חזרה אל ה-Client.

לכן ה-Client אינו מניח שסדר קבלת התשובות הוא בהכרח סדר העדכניות שלהן.

8. מבנה השרת -

קוד השרת נמצא תחת:

io.github.tomerg12.fleetlink.server

ה-packages המרכזיים הם:

אחריות	Package
Accounts וסיסמאות	account
Sessions פעילים	session
התאמת שחקנים	matchmaking
חוקי ומצב המשחק	game
Coordination ו-sequencing	service
Deadlines	deadline
Rematch negotiation	rematch
Elo ו-live rating	rating
Leaderboard ו-Statistics, History	statistics
Persistence ו-JPA Entities	persistence
שמירת תוצאות משחק	completion
remote implementation ו-RMI runtime	rmi

RMI Runtime 8.1

מחלקות עיקריות:

- FleetLinkServerMain.
- FleetLinkServerRuntime.
- FleetLinkServerRemoteImpl.

FleetLinkServerMain הוא entry point של השרת.

FleetLinkServerRuntime יוצר ומחבר את רכיבי השרת.

FleetLinkServerRemoteImpl מממש את FleetLinkServerRemote.

ברירת המחדל היא:

- RMI port: 1099
- Binding: FleetLinkServer

9. ניהול משתמשים ו-Sessions -

9.1 משתמש רשום

עבור משתמש רשום נשמרים גם username וגם username_key. username שומר את שם המשתמש כפי שהוא מוצג למשתמש, ואילו username_key הוא ערך מנורמל שמשמש לזיהוי וחיפוש של שם המשתמש ללא תלות באותיות גדולות או קטנות. נשמרים גם עבור משתמש רשום נתוני אימות סיסמה ו-Rating. סיסמה אינה נשמרת כטקסט רגיל. PasswordHasher משתמש ב:

- PBKDF2WithHmacSHA256.
- 600,000 iterations.
- 16-byte random salt.
- 256-bit derived key.

בבסיס הנתונים נשמרים:

- password_hash.
- password_salt.
- password_iterations.

9.2 משתמש אורח

Guest אינו יוצר חשבון קבוע.

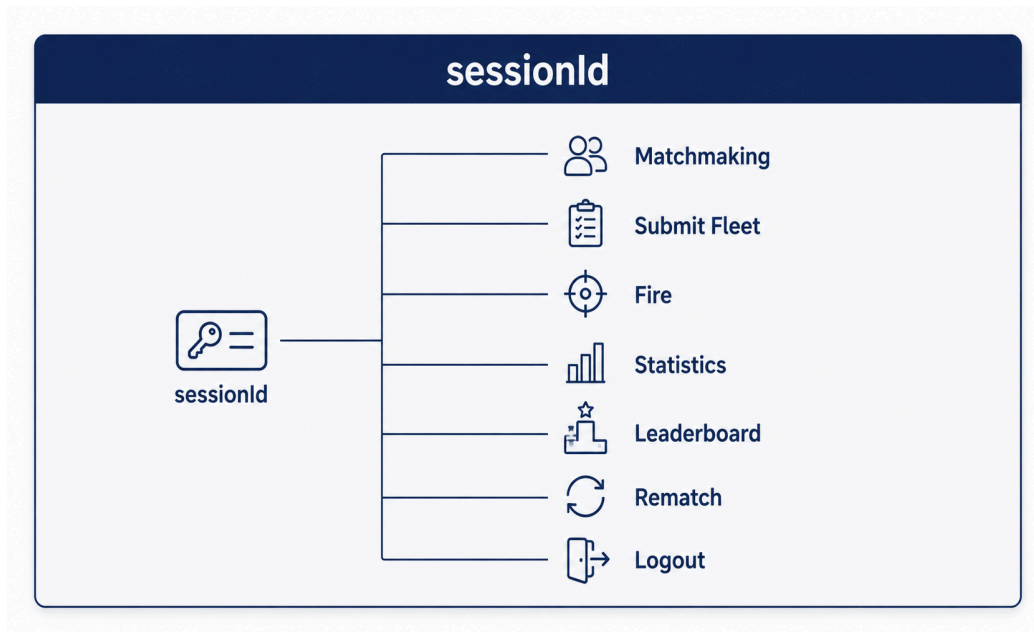
הוא מקבל:

- Session זמני.
- UUID זמני.
- Display Name.
- Matchmaking Rating זמני של 1000.

Guest אינו מקבל Rating קבוע ואינו מקבל Personal Statistics או Match History. הוא כן יכול לצפות ב-Leaderboard הציבורי.

Session 9.3

אחרי login, register או connectAsGuest השרת מחזיר sessionId.



איור 5: שימוש ב-sessionId - הפעולות המרכזיות שניתן לבצע לאחר יצירת Session תקין.

Session נשמר בזיכרון בלבד.

סיום Session מבטל מיד את היכולת להשתמש בו לפעולות חדשות.

ניקוי מצב השחקן מתבצע כך ש-Callbacks או פעולות ישנות לא יוכלו להשתלט מחדש על Session חדש של אותו משתמש.

- Matchmaking .10

MatchmakingService מנהל את המשתמשים שמחכים למשחק.

הטווח המרבי בין Ratings הוא:

• 200

המערכת מחפשת את המשתמש הממתין בעל ה-Rating הקרוב ביותר בתוך הטווח.

אם קיימים שני מועמדים במרחק Rating זהה, המשתמש שנכנס קודם לתור ההמתנה מקבל עדיפות.

Guest משתמש לצורך Matchmaking ב:

• 1000

הטווח אינו מתרחב עם זמן ההמתנה.

כאשר נמצא זוג מתאים:

• שני המשתמשים מוסרים מתור ההמתנה.

• נוצר GameSession.

• המשחק מקבל UUID.

• המשחק מופעל לשלב Ship Placement.

• השרת שולח onMatchFound לשני הלקוחות.

אין polling חוזר מצד ה-Client לחיפוש Match.

onMatchFound הוא מנגנון העדכון הרגיל כאשר משחק נוצר.

Rematch אינו עובר דרך סינון ה-Rating של Matchmaking רגיל.

11. מנוע המשחק -

GameSession 11.1

GameSession מחזיק את מצב המשחק authoritative.
הוא כולל בין היתר:

- 2 players.
- Boards.
- Ships.
- Game Phase.
- Current Turn.
- Shots.
- Timeout strikes.
- Deadline.
- Winner.
- End Reason.
- Telemetry.

Board 11.2

Board מנהל את מצב הלוח ואת תוצאות היריות.
הלוח הוא בגודל 10 על 10.

FleetValidator 11.3

לפני שהצי מתקבל, השרת בודק את ההצבה.
הבדיקות כוללות:

- כל הספינות הנדרשות קיימות.
 - הספינות נמצאות בתוך הלוח.
 - אין חפיפה.
 - האורך והכיוון תואמים ל-Ship Type.
- ה-Client שולח את כל הצי בפעולה אחת:

`submitFleet(sessionId, placements)`

אין RMI call נפרד לכל ספינה.

Battle 11.4

ירייה מתבצעת באמצעות:

`fire(sessionId, coordinate)`

השרת בודק:

- Session תקין.
- משחק פעיל.
- Phase תקין.
- Turn ownership
- Deadline
- Coordinate תקין.
- שלא נורתה כבר ירייה לאותו תא.

רק לאחר שהפעולה אושרה השרת קובע HIT או MISS ומשנה את מצב המשחק.
ה-Client אינו מנבא פגיעה, החטאה, מעבר תור או ניצחון.

12. מגבלות זמן ו-AFK -

הזמן הוא Server Authoritative.

Ship Placement 12.1

כאשר המשחק מופעל, שני המשתתפים מקבלים אותו deadline:

• 120 seconds

שליחת צי אינה מאריכה את הזמן.

אם רק שחקן אחד שלח צי עד סוף הזמן:

FINISHED

TIMEOUT

winner = player who submitted the fleet

אם אף שחקן לא שלח צי:

FINISHED

NO_CONTEST

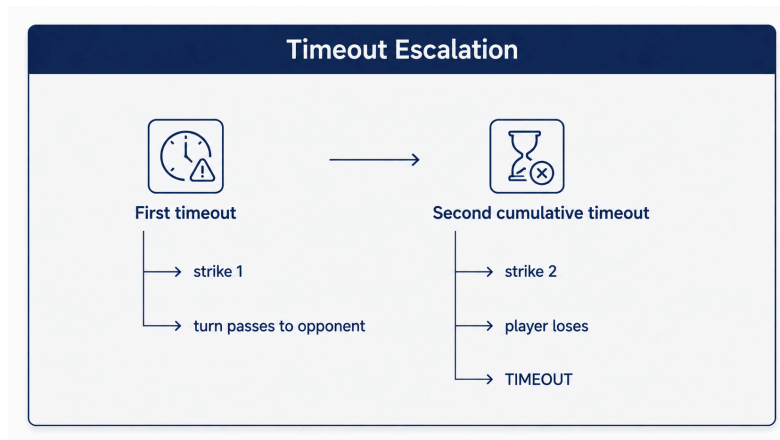
winner = null

:NO_CONTEST

- אינו Win.
- אינו Loss.
- אינו Rated.
- אינו נשמר בבסיס הנתונים.

Battle 12.2

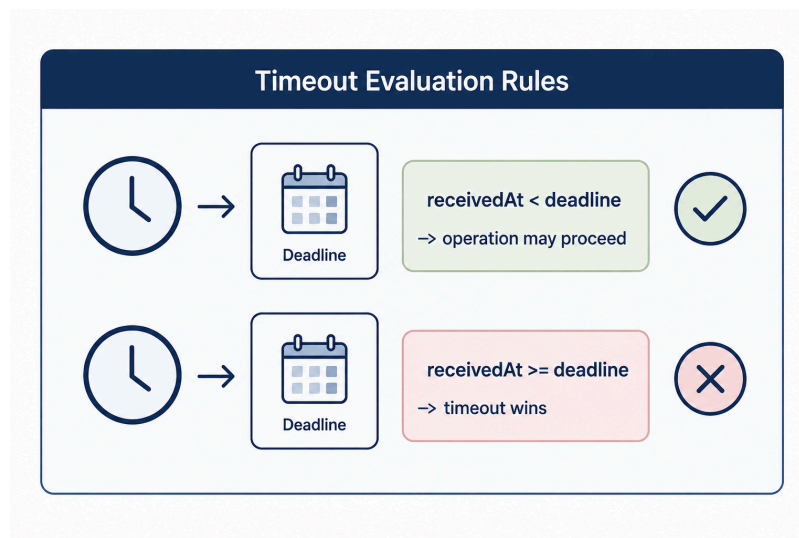
כל Turn מקבל 45 שניות. Timeout strikes מצטברים לאורך המשחק ואינם מתאפסים לאחר פעולה חוקית.



איור 6: מנגנון Timeout ב-Battle - הצטברות Timeout strikes והפסד לאחר ה-timeout המצטבר השני.

Deadline Race 12.3

כאשר פקודת משתמש נכנסת ל-GameCommandSequencer, השרת שומר את זמן הקליטה שלה ב-receivedAt לפי שעון השרת. אם $\text{receivedAt} < \text{deadline}$, הפעולה יכולה להמשיך לבדיקה. אם $\text{receivedAt} \geq \text{deadline}$, ה-timeout מקבל עדיפות. שעון ה-Client משמש רק להצגת ה-Countdown ואינו קובע את תוצאת מרוץ הזמן.



איור 7: הכרעה במרוץ מול ה-Deadline - פעולה מתקבלת רק כאשר זמן הקליטה שלה מוקדם מה-deadline.

13. סיום משחק ושמירת תוצאה -

סיבות הסיום הן:

- ALL_SHIPS_SUNK.
- RESIGNATION.
- DISCONNECT.
- TIMEOUT.
- NO_CONTEST.

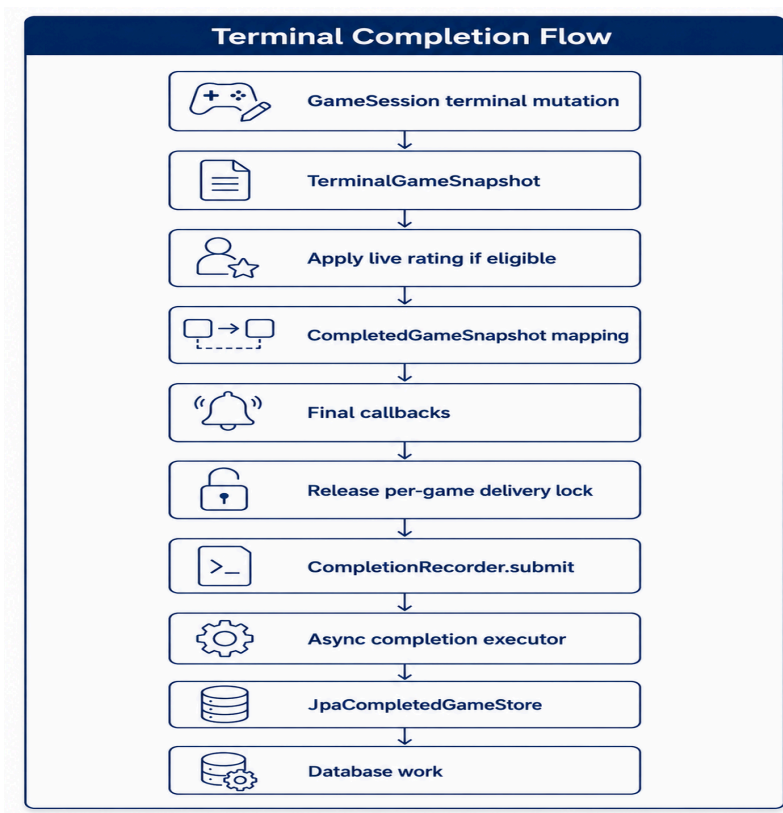
Forfeit של המשתמש ממומש בצד הפרוטוקול באמצעות:

`leaveGame(sessionId)`

והתוצאה authoritative היא:

RESIGNATION

במעבר סופי שמייצר Winner, הזרימה העקרונית היא:



איור 8: תהליך סיום משחק - יצירת Snapshot סופי, עדכון Rating, שליחת Callbacks והעברת התוצאה לשמירה אסינכרונית.

פעולת השמירה לבסיס הנתונים אינה מתבצעת בזמן שה-GameSession monitor מוחזק, והיא גם אינה מתבצעת כחלק מהנתיב הסינכרוני של פקודת המשחק. לאחר סיום ה-Callbacks ושחרור נעילת ה-delivery של המשחק, ה-Snapshot מועבר ל-CompletionRecorder, שמבצע את השמירה בצורה אסינכרונית.

משתתפים	נשמר
Registered vs Registered	כן
Registered vs Guest	כן
Guest vs Registered	כן
Guest vs Guest	לא
NO_CONTEST	לא

משחק Mixed יכול להופיע בהיסטוריה ובסטטיסטיקות של השחקן הרשום, אך הוא אינו Rated.

- Rematch .14

RematchCoordinator מנהל את ה-Rematch.

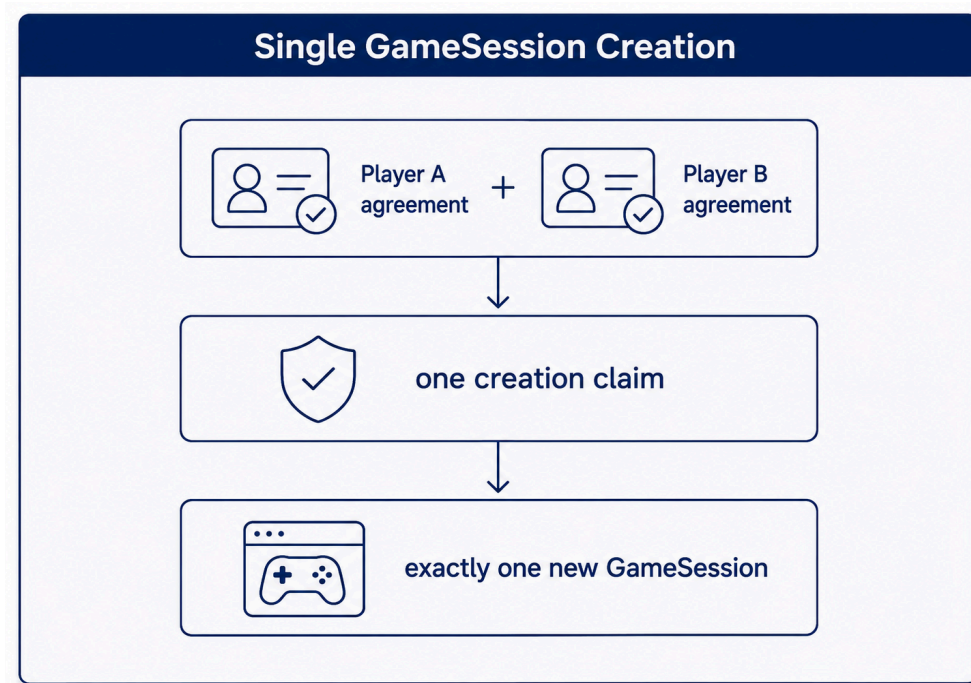
המשא ומתן נשמר בזיכרון בלבד ומקושר למשחק שהסתיים ולשני המשתתפים המדויקים שלו.
הפעולות הן:

`requestRematch(sessionId)`

`respondToRematch(sessionId, boolean accept)`

`respondToRematch(false)` משמש גם:

- לסירוב להצעת היריב.
- למשיכת בקשת Rematch שהשתמש עצמו כבר שלח.
- לסגירת הזדמנות Rematch כאשר חוזרים ל-Lobby.
- משחק חדש לא נוצר עד ששני השחקנים מסכימים.



איור 9: יצירת Rematch - משחק חדש נוצר רק לאחר הסכמה של שני המשתתפים, ובכל מקרה נוצר GameSession אחד בלבד.

בקשה זוהי שחוזרת שוב אינה חייבת לשנות את המצב פעם נוספת. כך למשל לחיצה חוזרת על אותה בקשת Rematch אינה יוצרת משחק נוסף. גם אם שני הצדדים מאשרים במקביל, נוצר לכל היותר משחק חדש אחד.

Callback failure אינו מבטל Rematch שכבר נרשם ואינו מבטל משחק שכבר נוצר.
לאחר יצירת המשחק החדש, המעבר ל-Ship Placement מתבצע באמצעות:

onMatchFound(newGame)

המשחק החדש הוא משחק FleetLink רגיל ומקבל Game ID חדש ו-Placement Deadline חדש.
Rematch אינו נשמר לאחר restart של השרת.

15. מערכת הדירוג -

משתמש רשום חדש מתחיל ב:

• Rating = 1000

המערכת משתמשת ב-Elo.

עבור שחקן A:

expectedA =

$$1 / (1 + 10^{((\text{ratingB} - \text{ratingA}) / 400)})$$

לאחר המשחק:

newRating =

$$\text{round}(\text{rating} + 32 * (\text{score} - \text{expected}))$$

כאשר:

- K = 32
- Win score = 1
- Loss score = 0

Rating אינו יכול לרדת מתחת ל-0.

אין Maximum Rating שהוגדר כחלק ממדיניות המשחק.

משחק Rated דורש ששני המשתתפים יהיו Registered.

סיבות Rated הן:

- ALL_SHIPS_SUNK.
- RESIGNATION.
- DISCONNECT.
- TIMEOUT.

אינם Rated:

- Registered vs Guest.
- Guest vs Registered.
- Guest vs Guest.
- NO_CONTEST.

Live Rating 15.1

RegisteredRatingRegistry הוא מקור ה-Rating העדכני בזיכרון בין סיום משחק לבין השמירה האסינכרונית בבסיס הנתונים. אם עדיין אין למשתמש רשום ערך בזיכרון, ה-Live Rating מאותחל מה-Rating הקבוע של החשבון. Logout אינו מוחק את הערך הזה. Restart של השרת מנקה את ה-registry, ולכן לאחר ההפעלה מחדש בבסיס הנתונים משמש שוב כמקור ההתחלתי. שינוי ה-Rating של שני השחקנים במשחק מדורג מתבצע ב-live state לפני שליחת ה-terminal callbacks.

Durable Rating Integrity 15.2

לכל Registered Player נשמר גם rating_revision. במשחק מדורג נשמרים:

- rating_at_match.
- rating_delta.
- rating_revision_before.

בעת Commit קבוע השרת מוודא שה-Rating וה-revision בבסיס הנתונים עדיין תואמים למצב שעל בסיסו חושב העדכון. כך משחק שכבר נשמר לא יכול לשנות Rating פעם שנייה ועדכוני משחקים אינם מיושמים בסדר שגוי.

16. סטטיסטיקות, היסטוריה וטבלת מובילים -

Player Statistics 16.1

PlayerStatisticsView כולל:

- Current Rating.
- Total Games.
- Wins.
- Losses.
- Win Rate.
- Ships Sunk.
- Total Shots.
- Hits.
- Accuracy.
- Average Hits per Game.

Averages ו-Ratios נגזרים מהנתונים ואינם נשמרים כעמודות נפרדות.

Current Rating מגיע מה-live rating של התהליך.

יתר הנתונים וה-Match History מחושבים ממשחקים שכבר נשמרו בהצלחה בבסיס הנתונים.

Match History 16.2

כל רשומה יכולה לכלול:

- Opponent Display Name.
- Is the opponent guest.
- WIN or LOSS.
- End Reason.
- Turns Taken.
- Duration.
- Accuracy.
- Ships Sunk.
- Rating Delta.
- Completed At.

ה-History תומך ב-pagination בצד השרת.

ה-RMI contract מאפשר לבקש עד 50 history rows בבקשה אחת.

Leaderboard 16.3

ה-Leaderboard הוא ציבורי לכל Session תקין. Guest יכול לצפות בו. הוא כולל Registered Players בלבד. ה-RMI contract מאפשר לבקש עד 100 leaderboard rows בבקשה אחת. המיון וה-limit מבוצעים בצד בסיס הנתונים.

Lobby Preview 16.4

ב-Lobby של Registered Player נטענים:

- 3 recent history rows.
- 5 leaderboard rows.

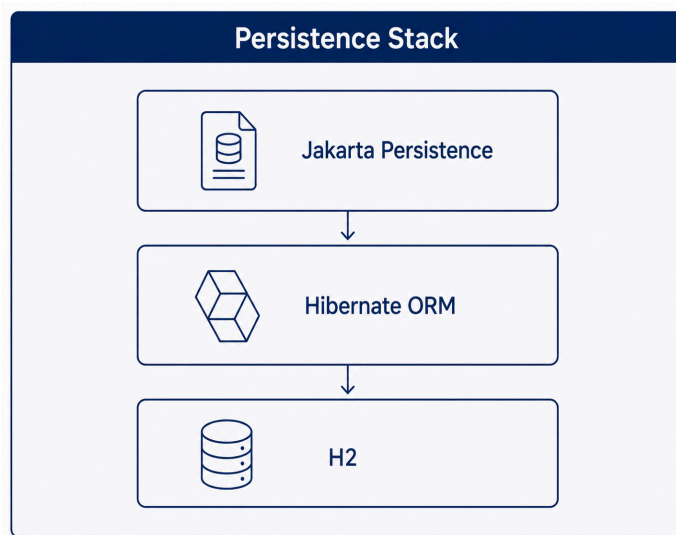
ומוצגים גם:

- Live Rating.
- Games.
- Wins.
- Win Rate.

Lobby של Guest אינו שולח בקשת Personal Statistics, אך כן מבקש 5 leaderboard rows.

17. שכבת Persistence -

שכבת ה-Persistence קיימת רק ב-Server. ה-FleetLinkPersistence מחזיק one process-wide EntityManagerFactory. כל פעולת Persistence יוצרת EntityManager משלה ומבצעת Transaction מתאים. ה-Persistence Unit נקרא fleetlink ומוגדר: RESOURCE_LOCAL ה-stack הוא:



איור 10: שכבת ה-Persistence - השימוש ב-Hibernate ORM, Jakarta Persistence, ו-H2 לצורך שמירת הנתונים.

כתובת ה-DB:

`jdbc:h2:file:./data/fleetlink`

Hibernate מוגדר עם:

`hibernate.hbm2ddl.auto = update`

המערכת אינה מוחקת את בסיס הנתונים בכל Startup.

18. בסיס הנתונים -

הישויות הקבועות הן:

- PlayerEntity.
- CompletedGameEntity.
- GameParticipantEntity.

players 18.1

תיאור	Field
UUID, Primary Key	id
שם המשתמש לתצוגה	username
זהות מנורמלת ו-Unique	username_key
Password hash	password_hash
Password salt	password_salt
PBKDF2 iterations	password_iterations
Rating קבוע	rating
Rating של Revision	rating_revision
זמן יצירת החשבון	created_at

קיים Leaderboard index:

- rating DESC.
- username_key ASC.
- id ASC.

completed_games 18.2

תיאור	Field
UUID של המשחק	id
זמן activation של המשחק	started_at
זמן הסיום	completed_at
סיבת הסיום	end_reason

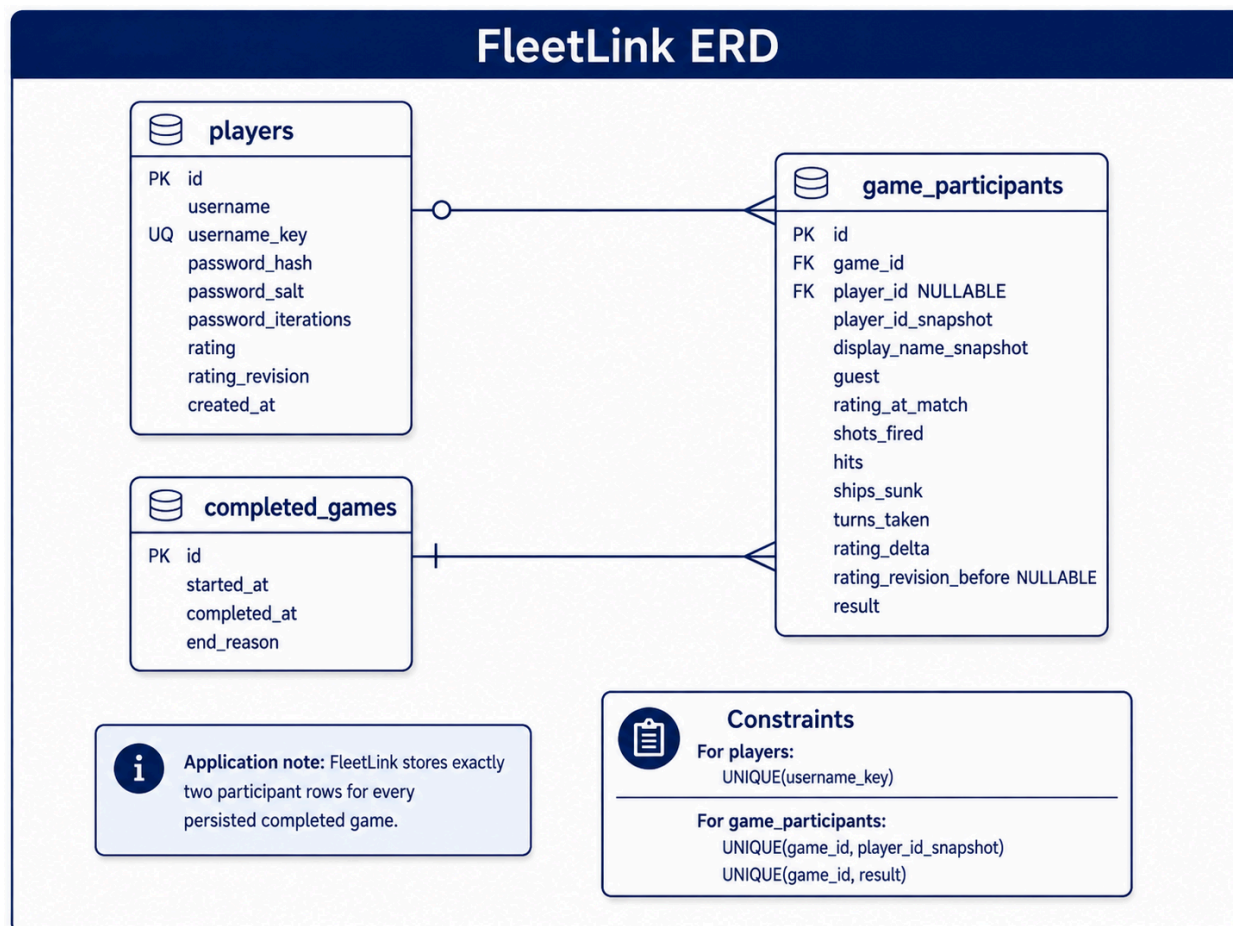
game_participants 18.3

תיאור	Field
UUID של הרשומה	id
קישור ל-Completed Game	game_id
קישור ל-Registered Player או NULL ל-Guest	player_id
מזהה המשתתף בזמן המשחק	player_id_snapshot
שם התצוגה בזמן המשחק	display_name_snapshot
האם המשתתף Guest	guest
Rating בתחילת המשחק	rating_at_match
Accepted shots	shots_fired
Hits	hits
Ships sunk	ships_sunk
Accepted shots + expired Battle turns	turns_taken
שינוי Rating	rating_delta
Revision לפני משחק Rated, אחרת NULL	rating_revision_before
WIN או LOSS	result

:Unique constraints

game_id, player_id_snapshot))
(game_id, result)

19. תרשים ERD -



איור 11: תרשים ERD - הישויות המרכזיות בבסיס הנתונים והקשרים בין players, completed_games ו-game_participants.

בסיס הנתונים כולל שלוש ישויות מרכזיות. PlayerEntity מייצגת חשבון רשום, CompletedGameEntity מייצגת משחק שהסתיים ונשמר, ו-GameParticipantEntity שומרת Snapshot של כל אחד מהמשתתפים במשחק. כל משחק שנשמר ב-FleetLink מכיל בפועל שתי רשומות Participant. עבור משתמש רשום, רשומת ה-Participant מקושרת גם לרשומה בטבלת players. עבור Guest הקישור player_id הוא NULL, אך player_id_snapshot ו-display_name_snapshot עדיין נשמרים כדי לשמור את פרטי המשתתף כפי שהיו בזמן המשחק.

20. מבנה הלקוח ו-JavaFX -

קוד ה-Client נמצא תחת:

io.github.tomerg12.fleetlink.client

שתי קבוצות מרכזיות:

- client.ui.
- client.integration.

client.ui 20.1

:Entry point

FleetLinkClientApplication.

מסכים:

- LoginScreen.
- LobbyScreen.
- ShipPlacementScreen.
- BattleScreen.
- GameOverScreen.
- PlayerStatisticsScreen.

אין מסך Registration נפרד.

LoginScreen תומך בשלוש פעולות:

- Login.
- Register.
- Continue as Guest.

Guest משתמש באותו Username field כ-Display Name ואינו צריך Password.

הניווט משתמש ב:

- ScreenNavigator.
- ScreenId.

Presentation classes מרכזיות:

- LobbyPresentationModel.
- ShipPlacementPresentationModel.
- BattlePresentationModel.
- GameOverPresentationModel.
- StatisticsPresentation.

העיצוב המשותף נמצא ב:

fleetlink.css.

client.integration 20.2

מחלקות מרכזיות:

- RmiClientGateway.
- ClientCallbackEndpoint.
- ClientOperationService.
- ClientStateCoordinator.
- ClientUiDispatcher.
- RmiClientConfig.
- RematchClientState.
- StatisticsDashboardState.

RmiClientGateway עוטף את הקריאות ל-RMI.

ClientOperationService מריץ remote operations מחוץ ל-JavaFX Application Thread.

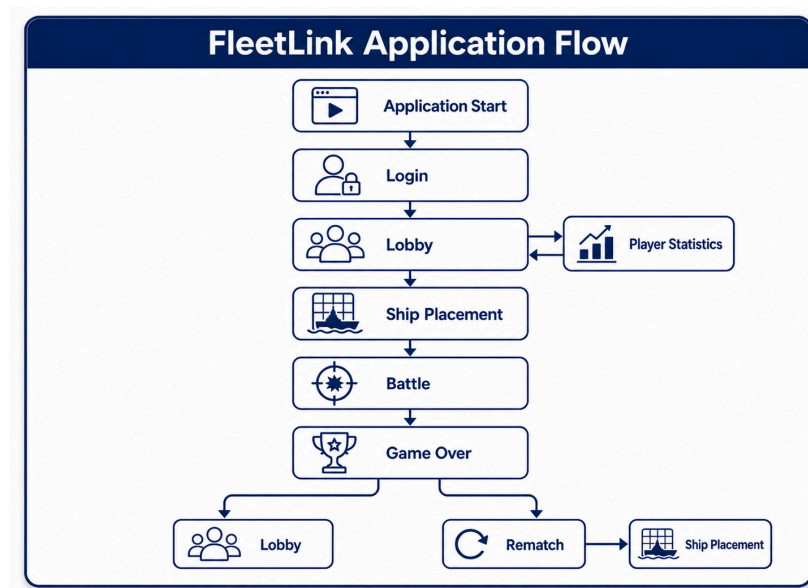
ClientCallbackEndpoint מקבל Callbacks.

ClientStateCoordinator מבצע reconciliation בין Session, Callbacks, Results ו-Game lifecycle.

ClientUiDispatcher מעביר presentation work אל JavaFX Application Thread.

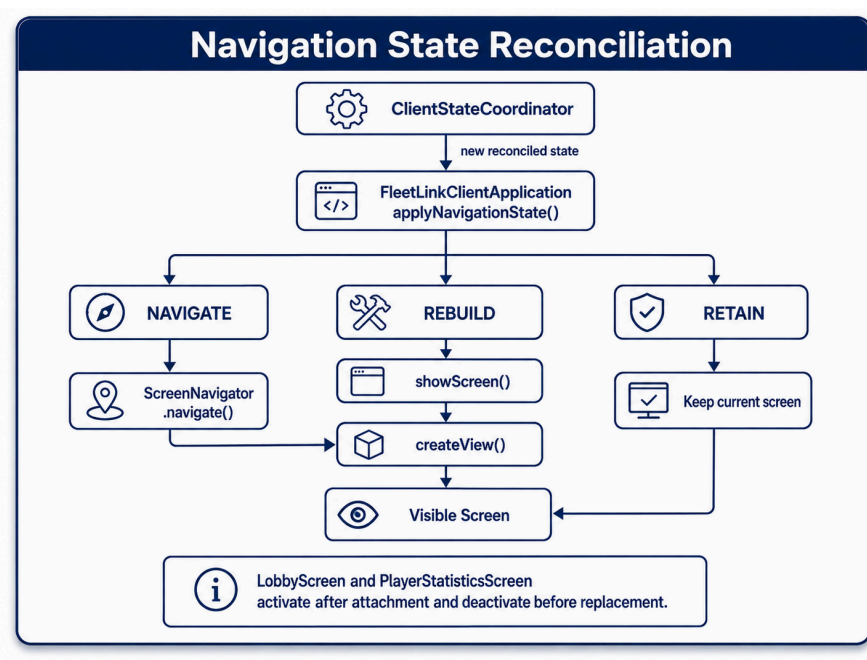
21. Screen Lifecycle ו- Navigation -

הזרימה המרכזית היא:

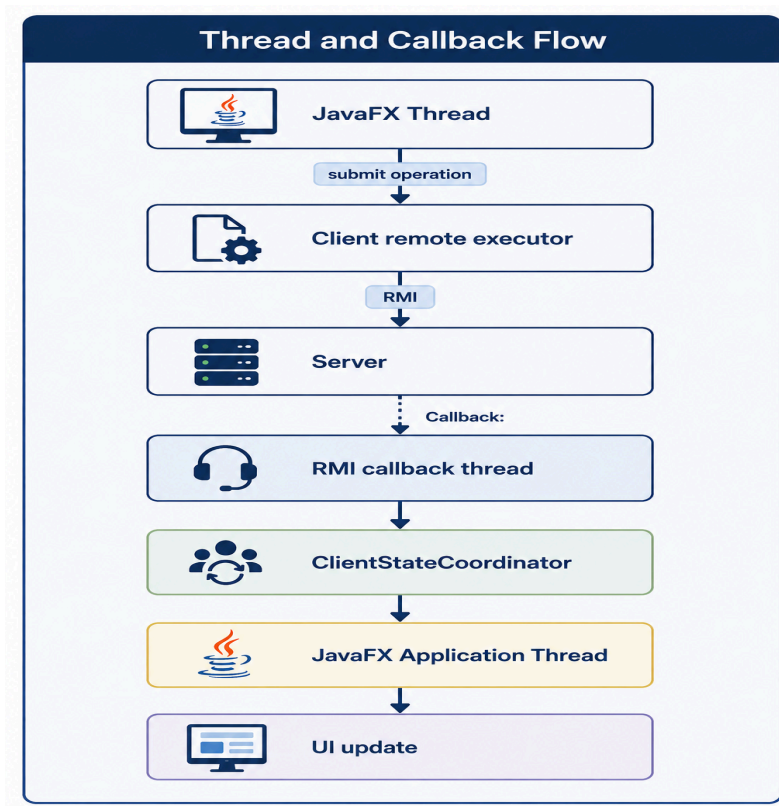


איור 12: זרימת האפליקציה - המעבר בין מסכי Login, Lobby, Player Statistics, Ship Placement, Battle, Game Over ו-Rematch.

איור 13: מנגנון ה-Navigation - בחירת הפעולה NAVIGATE, REBUILD או RETAIN בהתאם למצב החדש שהתקבל מהשרת. ה-Screen lifecycle הרעיוני הוא:



Blocking RMI calls אינם מתבצעים ב-JavaFX Application Thread.



איור 14: זרימת Threads ו-Callbacks ב-Client - ביצוע פעולות RMI מחוץ ל-JavaFX Application Thread והחזרת עדכוני UI אליו.

Callback state נרשם לפני תזמון עבודת presentation.
 כך Result ישן של פעולה אינו יכול להחזיר את ה-Client למצב שכבר הוחלף על ידי Callback חדש יותר.

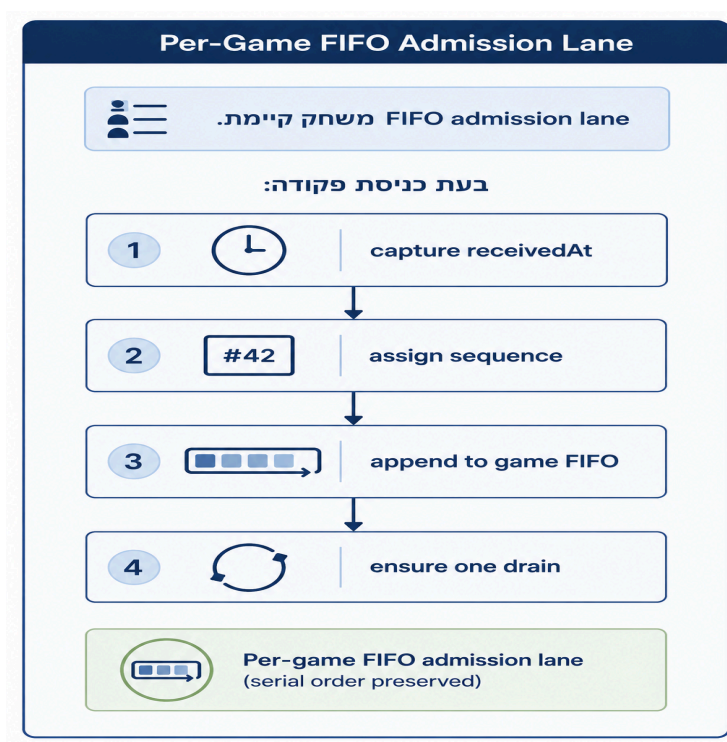
22. Concurrency וניהול מספר משחקים

במקביל -

אחת מדרישות המערכת היא שכמה זוגות שחקנים יוכלו לשחק בו זמנית. לכל משחק יש GameSession נפרד. GameSessionManager מחזיק את המשחקים הפעילים.

GameCommandSequencer 22.1

לכל משחק קיימת FIFO admission lane. בעת כניסת פקודה:

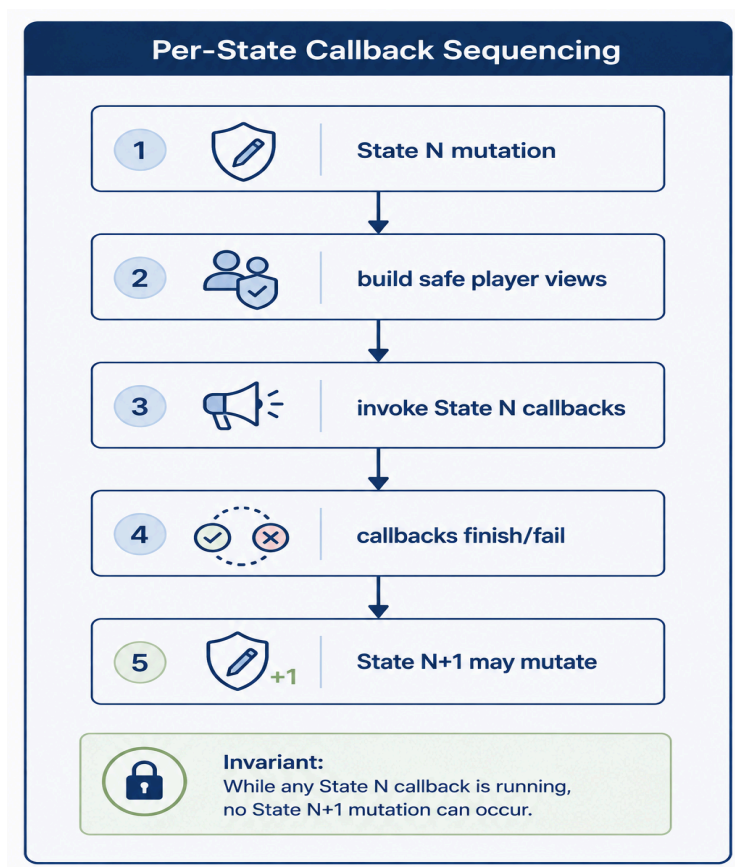


איור 15: תור פקודות לכל משחק - קליטת הפקודות וסידורן ב-FIFO לפני ביצוען על ידי Drain יחיד לכל Game.

לכל Game יש לכל היותר Drain פעיל אחד. ה-Drains רצים על Virtual Threads. משחקים שונים משתמשים ב-lanes נפרדים ולכן יכולים להתקדם במקביל.

Callback Ordering 22.2

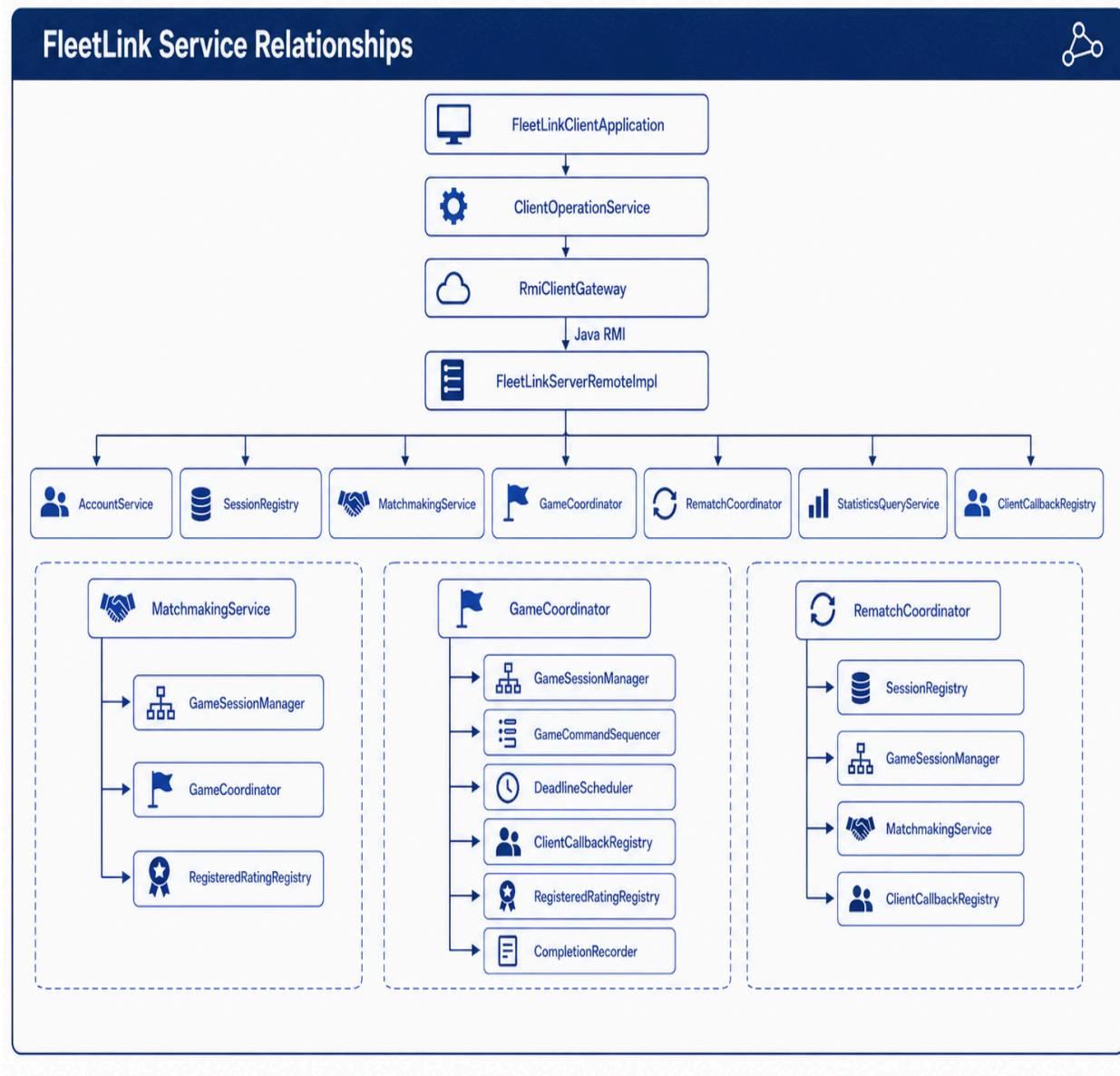
עבור אותו Game:



איור 16: סדר ה-Callbacks לכל מצב משחק - מצב חדש אינו יכול להשתנות שוב לפני סיום ניסיונות ה-Callbacks של המצב הנוכחי.

Callbacks לשני המשתתפים עבור אותו state יכולים להתחיל במקביל. RMI Callback אינו מבוצע כאשר ה-GameSession monitor מוחזק. משחק אחד עם Callback איטי אינו חוסם משחק אחר. עם זאת, Callback שאינו חוזר יכול לעכב את אותו Game, משום שאין Callback timeout ייעודי.

23. קשרים בין המחלקות המרכזיות -



איור 17: קשרים בין המחלקות המרכזיות - הקשרים בין רכיבי ה-Client, ממשק ה-RMI והשירותים המרכזיים של ה-Server.

24. מחלקות ומתודות עיקריות -

Shared 24.1

FleetLinkServerRemote: ה-Interface שמגדיר את פעולות ה-Client מול ה-Server. מתודות עיקריות:

- login().
- register().
- connectAsGuest().
- logout().
- joinMatchmaking().
- cancelMatchmaking().
- submitFleet().
- fire().
- getCurrentGame().
- getPlayerStatistics().
- getLeaderboard().
- leaveGame().
- requestRematch().
- respondToRematch().

FleetLinkClientCallback: מתודות:

- onMatchFound().
- onGameStateChanged().
- onRematchRequested().
- onRematchStatusChanged().

GameView: ה-DTO המכיל Snapshot בטוח של Game state עבור שחקן מסוים.

Server 24.2

- FleetLinkServerMain**: ה- Entry point של השרת. מקים או משתמש ב-RMI Registry וקושר אליו את FleetLinkServer.
- FleetLinkServerRuntime**: אחראי על יצירת רכיבי השרת וחיבורם.
- FleetLinkServerRemoteImpl**: ה- Adapter שמקבל את פעולות ה-RMI ומעביר אותן לשירותים המתאימים.
- AccountService**: אחראי על:
- registration.
 - login.
 - account/session integration.
- SessionRegistry**: מחזיק ומנהל Sessions פעילים.
- MatchmakingService**: אחראי על תור ההמתנה, בחירת יריב ויצירת Match.
- GameSession**: מחזיק את מצב המשחק authoritative.
- GameSessionManager**: מחזיק את אוסף המשחקים הפעילים.
- FleetValidator**: בודק Fleet לפני קבלתו.
- GameCoordinator**: מתאם בין GameSession, Callbacks, Deadlines, completion ו-rating.
- GameCommandSequencer**: מסדר פקודות לכל Game ב-FIFO.
- ScheduledExecutorDeadlineScheduler**: מתזמן את רגעי ה-deadline. הוא אינו משנה בעצמו GameSession אלא מכניס expiration work ל-sequencer.
- RematchCoordinator**: מנהל את מצב ה-Rematch ומבטיח שגם כאשר שני המשתתפים מאשרים בו-זמנית, ייווצר לכל היותר משחק חדש אחד.
- EloRatingCalculator**: מחשב שינויי Elo.
- RegisteredRatingRegistry**: מחזיק את ה-live Rating של Registered Players.
- StatisticsQueryService**: מאמת את ה-Session ומפעיל את שאילתות ה-Statistics.
- StatisticsRepository**: מבצע pagination, ordering, limit ו-aggregation ב-DB.
- CompletionRecorder**: מקבל Snapshot של משחק שהסתיים ומעביר את פעולת השמירה ל-executor נפרד. במקרה של כשל זמני הוא שומר את המשימה בזיכרון ומנסה לבצע אותה שוב.
- JpaCompletedGameStore**: שומר Completed Game ואת Participant records באמצעות JPA.

Client 24.3

- FleetLinkClientApplication**: ה-Entry point של ה-JavaFX Client ובעל ה-navigation המרכזי.
- ScreenNavigator**: מבצע החלפת Screens.
- RmiClientGateway**: מבצע את ה-RMI calls בפועל.
- ClientOperationService**: מריץ פעולות Remote בצורה אסינכרונית מבחינת UI.
- ClientCallbackEndpoint**: מימוש ה-Client RMI Callback.
- ClientStateCoordinator**: מחזיק ומיישב את מצב ה-Client מול Results ו-Callbacks.
- ShipPlacementPresentationModel**: מחזיק את סידור הספינות המקומי לפני אישורו על ידי ה-Server.
- BattlePresentationModel**: ממיר GameView למצב תצוגה עבור Battle.
- GameOverPresentationModel**: ממיר Game result authoritative למידע של מסך הסיום.
- StatisticsDashboardState**: מחזיק את state של Statistics, History ו-Leaderboard requests.

25. טיפול בשגיאות ומצבים חריגים -

יש הפרדה בין Business Failure לבין RMI Transport Failure.

Business failures מוחזרים באמצעות Result DTOs.

דוגמאות:

- Credentials לא תקינים.
 - Session לא תקף.
 - Fleet לא חוקי.
 - Fire שלא בתור.
 - ירייה על תא שכבר נפתר.
 - Matchmaking operation במצב לא מתאים.
 - Guest שמבקש Personal Statistics.
 - Rematch שכבר אינו תקף.
 - RemoteException מייצג בעיה בשכבת התקשורת.
- ה-Client אינו מניח ש-Transport Failure פירושו בהכרח שהפעולה לא התבצעה בשרת.
ה-Server נשאר authoritative ולכן ה-Client אינו מייצר בעצמו תוצאה חלופית.

26. בנייה והרצת המערכת -

26.1 דרישות

לצורך בניית והרצת הפרויקט נדרש JDK 21. הפרויקט כולל Maven Wrapper, ולכן אין צורך בהתקנה נפרדת של Maven.
 הפקודות בדוגמאות הבאות מיועדות ל-Windows PowerShell. ב-Windows יש לוודא ש-JAVA_HOME מצביע על התקנת JDK 21.

26.2 בדיקת הפרויקט

```
.\mvnw.cmd clean verify
```

26.3 הרצת ה-Server

מתיקיית השורש:

```
.\mvnw.cmd -pl server -am package -DskipTests
```

```
$serverCp = Get-Content -Raw server\target\runtime-classpath.txt
```

```
java -cp "server\target\classes;shared\target\classes;$serverCp"  
io.github.tomerg12.fleetlink.server.rmi.FleetLinkServerMain
```

26.4 הרצת ה-Client

```
.\mvnw.cmd -pl client -am package dependency:build-classpath  
"-Dmdep.outputFile=target\runtime-classpath.txt" -DskipTests
```

```
$clientCp = Get-Content -Raw client\target\runtime-classpath.txt
```

```
java --module-path "$clientCp" --add-modules javafx.controls -cp  
"client\target\classes;shared\target\classes"
```


io.github.tomerg12.fleetlink.client.ui.FleetLinkClientApplication

Client נוסף מופעל באותה דרך מתוך PowerShell נוסף.

26.5 קבצי בסיס הנתונים

כאשר ה-Server מופעל מתיקייט השורש, בסיס הנתונים נמצא תחת:

data/fleetlink

הקובץ העיקרי הוא:

data/fleetlink.mv.db

27. בדיקות ואימות -

הפרויקט מכיל Unit Tests ו-Integration Tests בכל שלושת המודולים.
נבדקו בין היתר:

- Protocol DTOs.
- Serialization.
- RMI contracts.
- Board.
- Fleet validation.
- GameSession.
- Matchmaking.
- Concurrency.
- Deadlines.
- Sessions.
- Accounts.
- Password hashing.
- Persistence.
- Completion retry.
- Rating.
- Statistics.
- History.
- Leaderboard.
- Rematch.
- Server RMI integration.
- Client reconciliation.
- JavaFX presentation state.
- Real two-client integration.

בגרסת היישום הסופית:

385 tests passed

חלוקה:

shared: 86
server: 154
client: 145

בנוסף בוצעה בדיקה עם שני JavaFX Clients אמיתיים מול Server פעיל.

נבדקו בין היתר:

- .Login
- .Matchmaking
- .Ship Placement
- .Battle
- .MISS ו- HIT
- ניסיון ירייה חוזרת.
- .Deadlines
- .Forfeit
- .Game Over
- .Rematch
- .Statistics
- .Match History
- .Leaderboard
- Restart של ה-Server עם אותו H2 database.
- שמירת Accounts, Ratings ו-History לאחר Restart.

28. החלטות תכנון ומגבלות -

Server Authoritative 28.1

ה-Server בלבד מחזיק את המצב הקובע של המשחק.
Client-side validation מיועד לחוויית משתמש בלבד ואינו מחליף בדיקת Server.

Persisted Live State 28.2 אינו

המידע הבא נמצא בזיכרון:

- Sessions
- Matchmaking Queue
- Active Games
- Boards
- Current Turns
- Deadlines
- Rematch Negotiations

Persistent State 28.3

המידע הקבוע כולל:

- Registered Accounts
- Durable Ratings
- Eligible Completed Games
- Participant Telemetry
- Statistics-derived data
- Match History
- Leaderboard data

Active Game Recovery 28.4 אין

אם ה-Server קורס כאשר משחק פעיל, המשחק הפעיל אינו משוחזר לאחר Restart.
מידע שכבר נשמר כ-Completed Game נשאר קיים.

Persistence Retry 28.5

Completion retry קיים בזיכרון התהליך.
אין Durable Outbox

Callback Timeout 28.6

אין Callback Timeout או Heartbeat ייעודי.
Callback שלא חוזר יכול לעכב את ה-Game lane שלו, אך אינו אמור לעכב Games אחרים.

Matchmaking 28.7

הטווח הקבוע הוא:

• Rating points 200

אין Dynamic Range Expansion.

אפשרות כזו נשארה Future Work בלבד.

Database Schema Evolution 28.8

הפרויקט אינו כולל Migration Framework כגון Flyway או Liquibase.

Hibernate מוגדר עם

`hibernate.hbm2ddl.auto=update`

בסיס נתונים שנוצר בגרסת פיתוח ישנה ולא תואמת עשוי לדרוש reset ידני, אך בהפעלה רגילה של הסכמה הסופית הנתונים נשמרים בין הפעלות של השרת.

28.9 טכנולוגיות שלא נוספו

התקשורת בין ה-Client ל-Server מבוססת Java RMI, ולא נעשה שימוש ב-HTTP או WebSocket כפרוטוקול המשחק. תכונות נוספות כגון Chat, AI opponent ו-Spectator mode אינן חלק מהיקף הפרויקט.

29. סיכום -

FleetLink היא מערכת Client-Server מלאה למשחק צוללות בין שחקנים מרוחקים. המערכת מחלקת אחריות בצורה ברורה:

- JavaFX -> Presentation.
- RMI Server -> Business Logic and Authoritative Game State.
- JPA / Hibernate / H2 -> Persistence.

ה-Client אחראי על ממשק המשתמש ועל הצגת Snapshots שמגיעים מה-Server. ה-Server מנהל Accounts, Sessions, Matchmaking, Fleet Placement, Battle, מספר משחקים במקביל, Statistics, Deadlines, Forfeit, Rematch, Rating. ה-Persistence layer שומרת Accounts ונתונים של Games שהסתיימו כאשר הם זכאים לשמירה. החלוקה לשלושת המודולים:

- shared.
- server.
- client.

שומרת על גבולות ברורים בין Protocol, Server implementation ו-Client presentation. המערכת משתמשת ב-JavaFX, Java RMI ו-JPA-compatible Persistence כשלוש הטכנולוגיות המרכזיות של הפרויקט.